A Frozen-Protocol Evaluation of Execution Feedback in Repository-Held-Out Coding-Agent Traces

October 27, 2026

Abstract

Observable execution feedback may indicate that a coding agent is encountering difficulty, but whether such signals add predictive value beyond simple trajectory structure is an empirical question. This study evaluates the incremental predictive value of a single strictly defined test-failure signal in repository-held-out coding-agent traces. The analysis uses the pinned `nvidia/Open-SWE-Traces` snapshot at revision `c144d9d160dcd9f5ebcd9520d2d54111dda8b681`, comprising 207,489 trajectories across 2,654 repositories. The frozen protocol uses trajectory-level outcomes, a five-round prefix, L2-regularized logistic regression, and a paired repository-cluster bootstrap. The structural-only model achieved test average precision (AP) of 0.6409601282425617. Adding the binary indicator for a strict explicit test failure increased AP to 0.6410260199693942, for a difference of 0.00006589172683257871. The recorded paired 95% bootstrap interval was [0.00005607830095563 65, 0.00013282286584345433], based on 1,000 valid replicates. The issue-weighted sensitivity difference was similarly small at 0.00006719713546010642. The feature was extremely sparse: 72 of 207,489 trajectories were positive in the full snapshot, and only seven feature-positive trajectories entered the scored test set, across six repositories. Five of those seven trajectories ultimately resolved. The positive AP difference is reproducible under the frozen protocol, but its absolute magnitude and sparse test support do not establish practical value, stable generalization, or a general mechanism. The measurement evidence is based on AI-assisted adjudication of predicted positives from eight shards, not human-verified ground truth; the bootstrap interval does not include adjudication error. The study therefore closes with a small positive incremental AP result and practical value not established.

Keywords: coding agents; execution feedback; trajectory prediction; repository-held-out evaluation; average precision; AI-assisted adjudication

1 Introduction

Coding agents produce trajectories containing messages, tool calls, tool observations, and execution-related feedback. Structural properties of these traces can provide a baseline for predicting whether a trajectory ultimately resolves its associated issue. Execution feedback may add information beyond these structural properties, but the presence of a failure-like event is not equivalent to eventual task failure: an agent may observe a test failure and subsequently fix it.

This study evaluates the following research question, preserved verbatim from the frozen protocol:

> How well do published trajectory-reliability signals transfer to repository-level coding-agent traces, and do observable execution-feedback features add incremental predictive value beyond simple structural baselines under repository-held-out evaluation?

The confirmatory analysis tests whether one narrowly defined execution-feedback feature adds incremental predictive value beyond a frozen structural baseline. The feature is a binary indicator that an explicit test failure is observed within the first five completed rounds of a trajectory. It is not a composite of all failure-like events. In particular, exceptions, build failures, test setup or collection failures, unresolved package-level failure summaries, interruptions, unknown execution status, and exit-status/test-result disagreements are not merged into the primary feature.

The primary endpoint is

$$\Delta_{\mathrm{AP}} = \mathrm{AP}(\text{structural} + \text{strict explicit test failure}) - \mathrm{AP}(\text{structural-only}),$$

where average precision is operationalized using `sklearn.metrics.average_precision_score`. The positive outcome class is `resolved == 0`; `resolved == 1` is the non-failure class; and `resolved == -1` is excluded from supervised fitting and evaluation and counted separately. The unit of analysis is the trajectory, and discordant outcomes among trajectories associated with the same issue are not majority-voted.

The study is an independent predictive validation under a frozen protocol. It is not an exact reproduction of Holistic Trajectory Calibration (HTC), and it does not claim established novelty. The literature anchor remains unresolved; accordingly, the findings are framed as a bounded validation result rather than a comprehensive statement about prior work.

2 Methods

2.1 Dataset and outcome definition

The analysis uses the fixed snapshot of `nvidia/Open-SWE-Traces` at revision `c144d9d160dcd9f5ebcd9520d2d54111dda8b681`. The snapshot contains 207,489 trajectories in 84 parquet shards, associated with 2,654 repositories and 22,320 issue identifiers. The recorded outcome counts are 95,487 trajectories with `resolved == 0`, 65,244 with `resolved == 1`, and 46,758 with `resolved == -1`. Thus, 160,731 trajectories have known outcomes and are eligible for supervised fitting or evaluation.

There are 7,161 issues for which associated trajectories do not all share the same `resolved` value. These differences are retained at the trajectory level. No majority vote is used to collapse multiple trajectories into a single issue-level outcome. The analysis does not treat an issue as the unit of the primary outcome.

2.2 Repository-held-out split

The frozen split uses repository-connected components and seed `20261024`; outcome labels are not used to construct the split. The pre-filter split counts are shown in Table 1. No repository or issue crosses the training, validation, and test partitions. After excluding `resolved == -1`, the supervised dataset contains 160,731 known-label trajectories, including 20,118 scored test trajectories. The remaining 5,057 rows in the raw test partition have unknown outcomes and are not included in test scoring.

Table 1: Repository-held-out split before filtering unknown outcomes.

Partition	Trajectory rows	Repositories
Training	150,190	1,858
Validation	32,124	398
Test	25,175	398

2.3 Prefix checkpoint and predictors

The checkpoint rule was fixed before the confirmatory fit: use five completed rounds if at least 70% of known-label trajectories reach five rounds; otherwise use three. All 160,731 known-label trajectories reached five completed rounds, giving 100% coverage. The selected checkpoint was therefore five rounds.

The structural model uses the frozen minimal feature set: completed prefix rounds; whether the checkpoint was reached; assistant-message count; tool-call count; tool-observation count; distinct-tool count; counts for eight tool families (execute, edit, read, search, think, submit, other, and unknown); user-content character count; assistant-content character count; tool-observation character count; and tool-observation line count. The augmented model adds only `prefix_explicit_test_failure_any`, a binary indicator that a strict explicit test failure occurred within the five-round prefix.

The prefix-round count and checkpoint-reached indicator are constant in this snapshot: they equal five and one, respectively. They are retained as specified by the frozen feature definition but do not vary across these data. Repository identifiers, issue identifiers, trajectory identifiers, patches, `reasoning_content`, and information after the checkpoint are not predictors. Parser status and the full trajectory length are audit information rather than predictive features.

2.4 Operational definition and adjudication of execution feedback

The parser and feature dictionary were frozen at version v2. A strict explicit test failure requires an explicitly observed failed test case or test-summary failure. Build or compilation failures, test setup or collection failures, and unresolved package-level `FAIL` messages are kept separate because they do not necessarily establish that a test assertion failed. No composite `execution_failure` feature is constructed. Missing SWE-agent exit codes remain missing rather than being coded as zero. Non-zero exit status is excluded from the cross-scaffold primary model and was considered only in a secondary OpenHands analysis.

The measurement rule was informed by AI-assisted adjudication of predicted positives sampled from eight fresh shards, two per configuration, among shards not used in the earlier development samples. The strict explicit-test-failure sample contained 50 AI-adjudicated true positives out of 50. The recorded conditional one-sided 95% finite-population precision lower bound was 0.9419345961564922. For comparison, the exception signal had 47 true positives and three false positives among 50, with a recorded lower bound of 0.8522570532915361, and was excluded from the primary model. The OpenHands-only non-zero-exit signal had 50 of 50 AI-adjudicated true positives, with a recorded lower bound of 0.941889378774265, but remained secondary.

These precision bounds are conditional on the AI labels being correct and apply only to predicted positives in the eight selected shards. The adjudication is project evidence, not human-verified ground truth: reviewer labels remain blank and student verification is false. The sampling evidence

does not establish precision across all 84 shards, and the model-uncertainty interval reported below does not account for AI-adjudication error.

2.5 Model fitting and primary endpoint

Both models are L2-regularized logistic regression models without class weighting. The candidate regularization values are $C \in \{0.01, 0.1, 1.0, 10.0\}$, with a maximum of 5,000 iterations. Each model selects C by validation average precision, breaking ties in favor of the smaller value, and is then refit on the combined training and validation data. The test set is scored once. Both primary models selected $C = 10.0$.

The primary endpoint is the difference in test average precision between the augmented model and the structural-only model. Both models are evaluated on the same eligible test trajectories and labels. A paired repository-cluster bootstrap with seed `20261092` uses the same repository resamples for both model scores and requests 1,000 replicates. The reported interval is the 95% percentile interval over the paired differences. The post-run audit recorded 1,000 valid replicates and no skipped replicates.

As a sensitivity analysis, an issue-weighted AP is calculated within the eligible rows of the relevant partition using weight $1/n_j$ for each trajectory belonging to issue j, where n_j is the number of eligible known-label trajectories for that issue in that partition. This analysis does not replace the unweighted trajectory-level primary estimate.

3 Results

3.1 Primary test result

The primary results are shown in Table 2. The structural-only model achieved test AP of 0.6409601282425617. Adding the strict explicit-test-failure indicator yielded AP of 0.6410260199693942. The resulting difference was 0.00006589172683257871. The paired repository-cluster bootstrap 95% interval was [0.00005607830095563 65, 0.00013282286584345433], which lies above zero. All 1,000 requested bootstrap replicates were valid.

Table 2: Primary test average precision and paired difference.

Quantity	Recorded value
Structural-only test AP	`0.6409601282425617`
Structural plus strict test-failure test AP	`0.6410260199693942`
Primary Δ_{AP}	`0.00006589172683257871`
Paired repository-cluster bootstrap 95% interval	`[0.00005607830095563 65, 0.00013282286584345433]`
Bootstrap replicates (requested / valid / skipped)	1,000 / 1,000 / 0

The point estimate is positive under the frozen protocol, but its absolute magnitude is very small relative to a structural baseline near 0.641. The recorded result supports a small positive incremental AP difference; it does not by itself establish practical importance or stable generalization.

The issue-weighted sensitivity analysis was consistent in direction and magnitude. Structural-only issue-weighted AP was 0.657544906786695, compared with 0.6576121039221551 for the augmented

model, giving a difference of 0.00006719713546010642. This sensitivity estimate is not a replacement for the primary trajectory-weighted endpoint.

3.2 Sparsity of the primary feature

The primary feature was extremely sparse. It was positive for 72 of the 207,489 trajectories in the full snapshot. Of those, 55 had known outcomes and 17 had `resolved == -1`. The split-level counts are shown in Table 3. In the raw test partition, ten trajectories were feature-positive across seven repositories; three had unknown outcomes. Only seven feature-positive trajectories entered the scored test set, across six repositories.

Table 3: Feature-positive trajectory counts by partition.

Partition	Total positive	`resolved == 0`	`resolved == 1`	`resolved == -1`	Repositories
Training	57	15	28	14	31
Validation	5	2	3	0	5
Test (all labels)	10	2	5	3	7

Among the seven feature-positive scored test trajectories, five ultimately resolved and two did not. The repositories represented were `kedacore/http-add-on` (two trajectories), `Masterminds/semver`, `Qiskit/qiskit-terra`, `getkin/kin-openapi`, `go-kit/kit`, and `pion/rtp` (one trajectory each).

The mean predicted failure probability for these seven rows was 0.5615705816344531 under the structural-only model and 0.3290106124564764 under the augmented model; the augmented-model median was 0.3235934460350468. For the remaining 20,111 scored test rows, the corresponding means were 0.5921354644757796 and 0.5922173021405209. Thus, the augmented model changed predictions most visibly for a very small group of feature-positive rows, while the mean prediction for the rest of the test set was nearly unchanged. These descriptive summaries do not establish a general mechanism. In particular, the presence of an early test failure does not imply that the trajectory ultimately fails.

3.3 Descriptive configuration-level results

Table 4 reports average precision by configuration. These values are descriptive only; they were not the confirmatory endpoint and were not used to select the model or feature. Differences were positive in all four slices but remained small, ranging from approximately 8.5×10^{-6} to 1.19×10^{-4}.

Table 4: Descriptive test AP by configuration (not confirmatory endpoints).

Configuration	Rows	Structural AP	Augmented AP	Difference
OpenHands / Minimax	4,837	0.6253015632908716	0.6253435588682583	0.0000419955773867
OpenHands / Qwen	5,340	0.7221603596512849	0.7222205225420950	0.0000601628908101
SWE-agent / Minimax	5,518	0.5827631323614232	0.5828823362095297	0.0001192038481065
SWE-agent / Qwen	4,423	0.5793009104242453	0.5793094077613375	0.0000084973370902

These subgroup values are not independent confirmatory tests, do not establish a consistent practical benefit, and should not be used to select a new feature or endpoint after observing the results.

3.4 OpenHands-only secondary analysis

The historical OpenHands-only analysis compared the structural model with a model adding the OpenHands non-zero-exit indicator. The recorded structural AP was 0.6800000121048011, the augmented AP was 0.6789736160889954, and the historical difference was −0.001026396015805675. This direction differs from the primary result.

The row-level predictions for this secondary analysis were not saved. Accordingly, the post-run reconciliation could not independently recompute these values; the historical difference is reported as recorded and is not a verified re-computation. It remains secondary and is not combined with the primary endpoint. It also does not justify recoding missing SWE-agent exit statuses as zero or adding a composite execution-failure feature.

4 Discussion

4.1 Interpretation of the primary finding

Under the frozen protocol, adding the strict explicit-test-failure feature produced a positive test AP difference of 0.00006589172683257871, with a recorded paired repository-cluster bootstrap interval above zero. The difference is nevertheless very small against a structural-only AP of approximately 0.641. The most defensible interpretation is therefore narrow: the primary endpoint was positive under this particular split, fitted models, and frozen measurement rule, but the magnitude does not establish meaningful operational improvement.

The positive interval should not be read as a guarantee that the signal will help across repositories, datasets, coding-agent configurations, or future deployments. The bootstrap quantifies repository-resampling variability conditional on the fitted models and the frozen AI-assisted measurement rule. It does not include uncertainty about the adjudication labels themselves.

4.2 Sparse support and the distinction between process and outcome

The feature's sparsity is central to interpretation. Only 72 trajectories were positive across the full snapshot, and only seven feature-positive trajectories contributed to the scored test metric. These seven rows came from six repositories, and five ultimately resolved. The model lowered their mean predicted failure probability after adding the feature, while the average predictions for the other 20,111 scored rows were almost unchanged.

These observations are compatible with a localized change in ranking or scores, but they are too few to support a general mechanism. More fundamentally, a test failure observed during an early prefix is a process event, whereas `resolved` is a trajectory outcome. An agent may encounter a failed test and then repair the underlying problem. The fact that five of the seven feature-positive test trajectories resolved illustrates why an early failure event should not be treated as equivalent to eventual failure.

4.3 Measurement validity and scope

The strict feature was selected using an explicit event definition and AI-assisted adjudication of predicted positives. The recorded conditional one-sided lower bound exceeded the 0.90 threshold for

the strict test-failure signal. However, this result depends on the AI labels being correct and applies only to predicted positives sampled from eight fresh shards. It is not human verification and does not establish precision across all 84 shards.

The model evaluation and the feature-adjudication process answer different questions. The repository-cluster bootstrap characterizes variation in the measured predictive difference under resampling of repositories. It does not propagate possible AI-adjudication errors into that difference. The resulting interval should therefore be interpreted conditionally, not as a complete uncertainty interval for the entire measurement-and-modeling pipeline.

The analysis also preserves distinctions between observable event types. Exceptions, explicit test failures, build failures, setup or collection failures, package-level failure summaries, interruptions, unknown states, and exit-status/test-result discordance are not interchangeable. In particular, no composite `execution_failure` variable is introduced, and absent SWE-agent exit codes remain missing rather than being assigned a successful value.

4.4 Literature positioning and novelty boundary

The project record leaves the L1 literature anchor `UNRESOLVED`. Exact HTC reproduction is `NOT ACHIEVED`, and novelty is `NOT ESTABLISHED`. This paper therefore does not claim to be an exact HTC replication, a comprehensive literature review, or a demonstration that the proposed question is novel. Its contribution is limited to reporting the frozen-protocol incremental-prediction result and documenting its measurement and support limitations. A literature-complete submission would require a separately verified bibliography and resolution of the L1 anchor; neither is established by the supplied evidence files.

4.5 Limitations

Several limitations constrain the conclusions.

1. **Very sparse primary feature.** Only seven feature-positive trajectories entered the scored test set. This is insufficient to support claims of broad or stable practical usefulness.
2. **AI-assisted rather than human-verified adjudication.** The precision evidence assumes the AI labels are correct. The recorded bootstrap interval does not include adjudication error.
3. **Restricted precision-validation scope.** The adjudication sample covers predicted positives from eight selected shards, not all 84 shards. Universal feature precision is not established.
4. **Single pinned dataset snapshot and split.** The analysis uses one fixed dataset revision and one repository-held-out split. Generalization to other datasets or coding-agent configurations has not been established.
5. **Secondary-analysis reproducibility.** OpenHands row-level predictions were not saved, so its historical negative difference could not be independently recomputed in the post-run audit.
6. **Unresolved literature anchor.** L1 remains unresolved, exact HTC reproduction was not achieved, and novelty is not established.

5 Conclusion

This frozen-protocol study found a small positive incremental average-precision difference when a strict explicit-test-failure indicator was added to a structural baseline for repository-held-out coding-agent traces. The primary difference was 0.0000658917268325787 1, with a recorded paired repository-cluster bootstrap 95% interval of [0.0000056078300955636 5, 0.0001328228658434543 3]. The issue-weighted sensitivity difference was similarly small.

The result is conditional on the fitted models and AI-assisted measurement rule. The feature is extremely sparse, with only seven feature-positive scored test trajectories across six repositories; five of those trajectories ultimately resolved. The evidence does not establish broad practical value, stable generalization, universal precision across the dataset, exact HTC reproduction, or research novelty.

The project decision is therefore:

```
RESEARCH CLOSED -- SMALL POSITIVE INCREMENTAL AP;
PRACTICAL VALUE NOT ESTABLISHED
```

L1 remains `UNRESOLVED`, HTC exact reproduction is `NOT ACHIEVED`, novelty is `NOT ESTABLISHED`, and L3 is `NOT AUTHORIZED`.

Data and code availability

The dataset snapshot analyzed is identified as `nvidia/Open-SWE-Traces` at revision `c144d9d160dcd9f5ebcd9520d2d54111dda8b681`. The supplied project records document the frozen protocol, feature specification, saved predictions, metric reconciliation, and closeout decision. The supplied evidence does not establish a public release location for the analysis code or all derived artifacts; public availability is therefore not claimed.